Intent-Based Model Selection for Anomaly Detection using ns-O-RAN

Abdulla Ovais-CS24MTECH12014

Project Recap

Problem Statement

Choose the right detector from a small model catalog using operator intent and ns-O-RAN telemetry.

Motivation

Operators need a practical way to trade off accuracy, latency, and explainability without manual trial-and-error.

Key Objectives

Collect telemetry, train four models, build a catalog, and use a rule-based selector to recommend the best one.

The project stays lightweight, reproducible, and centered on one exp1 scenario.

Related Work / Positioning

Foundation: Existing works like ns-O-RAN and OpenRAN Gym establish telemetry flows, while O-RAN SC stacks define E2 integration.

Gap Addressed

We introduce an **intent-based selection layer** and a **measured model catalog** on top of standard telemetry workflows.

Key Contribution

Transforming raw traces into **KPIs**, training and cataloging models, and enabling **automated selection** based on operator intent.

Moving beyond collection to actionable, intent-driven model management.

System / Threat Model

System Model

ns-o-ran generates radio traces, Docker-based RIC provides O-RAN control, and Python processes telemetry into model-ready windows.

Key Assumptions

Fixed topology, UE count, traffic load, and random seed; only one handover knob varies per experimental run.

Threat Model

Focuses on misconfigured parameters or wrong detector choices; specifically excludes security attacks.

Scope

Validation of one exp1 (hosinrdiffrence) scenario to build a catalog for intent-based model selection.

Intent-driven model selection anchored in standard O-RAN telemetry workflows.

System Architecture / Methodology

Methodology

- Real ns-O-RAN / ns-3 setup for handover misconfiguration scenario.
- Raw traces (CellIdStats, DlE2Pdcp) converted to KPI features.

Models & Catalog

- Random Forest, Boosting, LogReg, SVM trained on same dataset.
- Performance saved in model catalog with accuracy/latency.

My Contribution

- Real telemetry collection & KPI window extraction.
- Model training, catalog creation & intent selection.

Selection Logic

- Rule-based selector uses operator intent for best model.
- Enables automated, intent-based model selection.

Flow: ns-3 traces → KPI windows → Model Training → Model Catalog → Intent-based Selection

Key Algorithm / Logic

1. Intent Definition

The operator gives an intent such as high accuracy, low latency, or simple baseline.

2. Intent Parsing

The selector parses the intent and maps it to a decision priority.

3. Catalog Comparison

It reads the saved model catalog and compares the measured metrics of each candidate model.

4. Model Selection

It selects the best model, returns the saved .pkl path, and generates a short explanation.

Automated, intent-driven model selection optimized for standard ns-O-RAN telemetry metrics.

Implementation Details

Hardware / Environment

Host-based ns-3 simulation, Docker-based RIC containers, and a Linux analysis workspace for offline model training.

Software & Tools

ns-3 / ns-O-RAN, Docker, Python 3, pandas, NumPy, scikit-learn, and the local intent-selector package.

Implementation Changes

Modified scenario.cc for handover misconfiguration; developed trace-export scripts, KPI-window builders, training scripts, catalog builder, and rule-based selector.

Implementation Details

What We Built vs. Reused

Reused ns-O-RAN simulator and RIC stack. Custom modifications for handover misconfiguration, raw trace collection (normal/anomaly), CSV conversion, model catalog, and selection logic.

Challenges

Overcame case-sensitive ns-3 globals, complex path handling across workspaces, and alignment between Linux and local folder structures.

Outcome

Reproducible end-to-end pipeline: From raw telemetry to an operator-intent model recommendation.

Experimental Setup

Setup & Reproducibility

Exp1 handover scenario; hoSinrDifference 3-18. Consistent topology, UE count, and traffic load.

Baselines

Random Forest, Gradient Boosting, Logistic Regression, and SVM.

Key Metrics

Accuracy, F1, ROC-AUC, latency, and handover-window KPI deltas.

Data Sources

CellIdStats, CellIdStatsHandover, and DlE2PdcpStats trace files.

Dataset & Training Samples

Normal/anomaly windows combined via catalog builder. 102 training and 35 test samples per model (stratified).

Results & Analysis

1) Proving the handover

Proving the handover and the anomaly caused by misconfiguration:

CellIdStatsHandover.txt contains the actual handover events. Each row signifies a UE changing cells at a specific time.

CellIdStatsHandover_anomaly.txt (Line 54 to 57)

6.31227 9 27 2

6.37327 9 22 5

6.37952 9 28 2

6.41327 9 23 5

This shows that the same UE (IMSI 9) with rnti number 27,22,28,23 first moved to **Cell 5** and then very quickly moved back to **Cell 2**. This fast back-and-forth behavior is a strong indicator of unstable handover behavior.

Results & Analysis

2) Proving the anomaly due to misconfiguration

In the handover-window summary:

handover_window_summary.csv (See in line 2)
loss_pct post-window: **normal 65.06%**, **anomaly 70.10%**

handover_window_summary.csv (See inline 3)
thr_rx_mbps post-window: **normal 123.45**, **anomaly 117.90**

This means:

- Packet loss **increased** in the anomaly case
- Received throughput **decreased** in the anomaly case

Conclusion: Since only hoSinrDifference was changed, this difference can be attributed to the handover misconfiguration.

Results & Analysis

```
(base) abdullaovais@user:~/Desktop/TWIN/ns-o-ran/results/scripts$ python3 select_exp1_model_by_intent.py \
  --catalog ns_oran_intent_selector/exp1_hosinr/model_catalog.json \
  --index ns_oran_intent_selector/exp1_hosinr/model_index.json \
  --intent "select the model for Dectecting anomaly  handover misconfiguration with high accuracy"
================================================================================

Intent: select the model for Dectecting anomaly  handover misconfiguration with high accuracy

Selected model: SVM

Reason: The intent prioritizes accuracy, so the model with the best measured classification score in the catalog is selected. Recommendation: Use the highest-scoring model for handover windows; catalog score is accuracy=0.771 and latency=0.028 ms.

Confidence: 0.93

Catalog accuracy: 0.7714

Catalog latency: 0.0279 ms

Model path: /home/abdullaovais/Desktop/TWIN/ns-o-ran/results/scripts/ns_oran_intent_selector/exp1_hosinr/model_outputs/svm/svm.pkl
```

Results & Analysis

```
(base) abdullaovais@user:~/Desktop/TWIN/ns-o-ran/results/scripts$ python3 select_exp1_model_by_intent.py \
  --catalog ns_oran_intent_selector/exp1_hosinr/model_catalog.json \
  --index ns_oran_intent_selector/exp1_hosinr/model_index.json \
  --intent "select the model for Dectecting anomaly  handover misconfiguration with low latency"
==========================================================================

Intent: select the model for Dectecting anomaly  handover misconfiguration with low latency

Selected model: Logistic Regression

Reason: The intent prioritizes speed, so the fastest model in the catalog is selected. Recommendation: Use the low-latency detector first; catalog score is accuracy=0.571 and latency=0.021 ms.

Confidence: 0.91

Catalog accuracy: 0.5714

Catalog latency: 0.0207 ms

Model path:
/home/abdullaovais/Desktop/TWIN/ns-o-ran/results/scripts/ns_oran_intent_selector/exp1_hosinr/model_outputs/logistic_regression/logistic_regression.pkl
```

Limitations / Failure Cases

Generalization Gaps

Validated only on handover misconfiguration and a small model catalog. May not generalize to unseen anomalies or varying network conditions.

Intent Ambiguity

Ambiguous operator intent or keyword mismatches can lead the selector to return a sub-optimal or less suitable model.

Simulated Telemetry

Trained on simulated ns-O-RAN data rather than full live deployments; real-world radio behavior may diverge from models.

Limitations & Failure Cases

Unrealistic Assumptions

- Fixed topology, UE count, and traffic load while changing one knob.
- Controlled experimental setup rather than a fully dynamic live network.

Scalability Issues

- Catalog limited to four models; expansion requires automation.
- Increased simulation/training costs for more UEs and anomaly classes.

Overhead

- Rule-based selector has very low runtime overhead.
- Primary costs are offline: data collection, parsing, and training.

Comparison with Objectives

Objective 1	Status (Completed/Partially Completed, Infeasible, etc)
Data collection	completed through normal and anomaly trace bundles.
build KPI windows	completed through handover-window CSV generation.
train and compare models	completed through four model scripts and test metrics.
build a model catalog	ompleted through model_catalog.json, model_results.csv, and model_index.json
rule-based intent selection	completed through the selector that maps intent to the best measured model.

Live Demo

- Scenario:In the live demo, I will show the full end-to-end ns-O-RAN workflow: starting the Docker-based RIC stack, running the normal and anomaly handover scenarios in ns-3, collecting the raw trace files, converting them into KPI windows, training the candidate models, building the model catalog, and finally selecting the best model based on an operator intent.
- (Switch to Live Demo / Show Backup Video here)

Live Demo

TWIN [SSH: 10.68.0.139]

Update

EXPLORER

- TWIN [SSH: 10.68.0.139]
  - dataset
  - flexric
  - ns_oran_intent_selector
  - ns-o-ran
    - colosseum-near-rt-ric
    - logs
    - ns-3-mmwave-oran
    - oran-e2sim
    - results
      - artifacts
      - exp
      - exp1
      - exp1_hosinr
      - exp2
      - exp2_hysteresis
      - exp3
      - exp3_TTT
      - scripts
        - __pycache__
        - ns_oran_intent_selector
          - __pycache__
          - exp1_hosinr
          - __init__.py
          - catalog.py
          - schema.py
          - selector.py
        - build_exp1_model_catalog.py 1
        - compute_kpis_from_pdcp_csv.py
        - exp1_training_utils.py
        - extract_ns_o_ran_kpis.py
        - fill_twin_report.py
        - handover_window_compare.py 1
        - select_exp1_model_by_intent.py
        - train_exp1_boosting.py
        - train_exp1_logistic_regression.py
- OUTLINE
- TIMELINE

selector.py | exp1_training_utils.py | select_exp1_model_by_intent.py | train_exp1_boosting.py | train_exp1_logistic_regressior

ns-o-ran > results > scripts > ns_oran_intent_selector > selector.py > ...

```
162 def _select_for_handover_exp1(intent: OperatorIntent, catalog: ModelCatalog) -> tuple[ModelSpec, str, str, float | None, str
177                 "handover_simple_baseline",
178                 "The intent asks for a simple baseline, so Logistic Regression is the lightest linear option in the catalog.
179                 0.88,
180                 "Use the linear baseline first, then compare it with the stronger non-linear models if the KPI pattern is su
181             )
182
183     if intent.interpretability_priority or intent.requires_labels:
184         selected = _pick_model_by_name_preference(
185             models,
186             ("Random Forest", "Logistic Regression", "SVM", "Gradient Boosting", "XGBoost"),
187         )
188         if selected.name == "Random Forest":
189             return (
190                 selected,
191                 "handover_labels_interpretability",
```

PROBLEMS 2 | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS 1

bash

```
(base) abdullaovais@user:~/Desktop/TWIN$
```

SSH: 10.68.0.139 | 0 2 1 | Ln 364, Col 1 | Spaces: 4 | UTF-8 | LF | Python | 3.10.18 | Python 3.10.18

Work Distribution

Abdulla Ovais

Project Lead

End-to-end Implementation

CORE CONTRIBUTIONS

- ns-O-RAN environment setup & data collection
- KPI export & machine learning model training
- Catalog build, selector logic & report generation

Conclusions

Summary of Key Contributions

I built an intent-based model-selection pipeline on top of real ns-O-RAN telemetry.

Impact of the Work

The operator can now choose a model based on accuracy, latency, or explainability, instead of manually picking one detector.

Future Work

I will try to do on real O-RAN testbed setup and apply controlled real anomalies and collect the data and make the intent based model selection for anomaly detection and RCA Using the LLM for the network operator

References

GitHub Repo Link
https://github.com/AbdullaOvais/TWIN-IITH_Intent-based-model-selection

ns-O-RAN paper
ns-O-RAN: Simulating O-RAN 5G Systems in ns-3

OpenRAN Gym ns-O-RAN tutorial
https://openrangym.com/tutorials/ns-o-ran

O-RAN SC sim-ns3-o-ran-e2
https://github.com/o-ran-sc/sim-ns3-o-ran-e2

e2sim
https://github.com/wineslab/o-ran-e2sim

Declaration on LLM Usage

LLMs Used

- OpenAI ChatGPT / Codex was used as a writing and coding assistant.
- Used exclusively for support tasks: explanation, drafting, and script cleanup.
- Core project logic remains rule-based and telemetry-driven.

Sample Prompts Used

- "Explain **hoSinrDifference**, **hysteresis**, and **time-to-trigger** in simple terms."
- "Help me convert raw ns-O-RAN trace files into KPI summaries."
- "Create a handover-window comparison script for normal vs anomaly runs."
- "Rewrite this slide content so it sounds natural and not too AI-like."

Declaration on LLM Usage

How LLM Output Was Validated

- Every generated script was tested on real trace files.
- Outputs were checked against raw logs and KPI summaries.
- The scripts were run locally before being used in the final workflow.
- Any code changes were kept small and reviewed manually.

How LLM Snippets Were Documented

- LLM-assisted code was kept in separate helper scripts.
- Comments and README notes were used to explain script functions.
- Analysis helpers are separated from the main ns-O-RAN setup.
- Final report and slides describe LLM as assistance only.